PLC Control Test No. 1 - Variant 1

Editable diagrams are available in PLC_CONTROL_TEST_variant_1_trees.drawio.

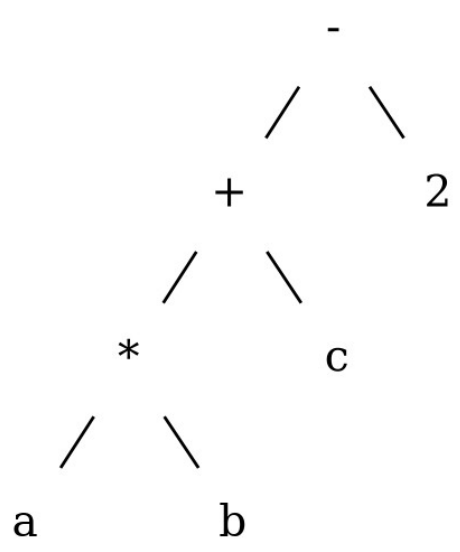
Task 1

Represent expressions in prefix and postfix notations and draw an abstract syntax tree.

Expression 1

a * b + c - 2

- Prefix notation: - + * a b c 2
- Postfix notation: a b * c + 2 -
- Abstract syntax tree:

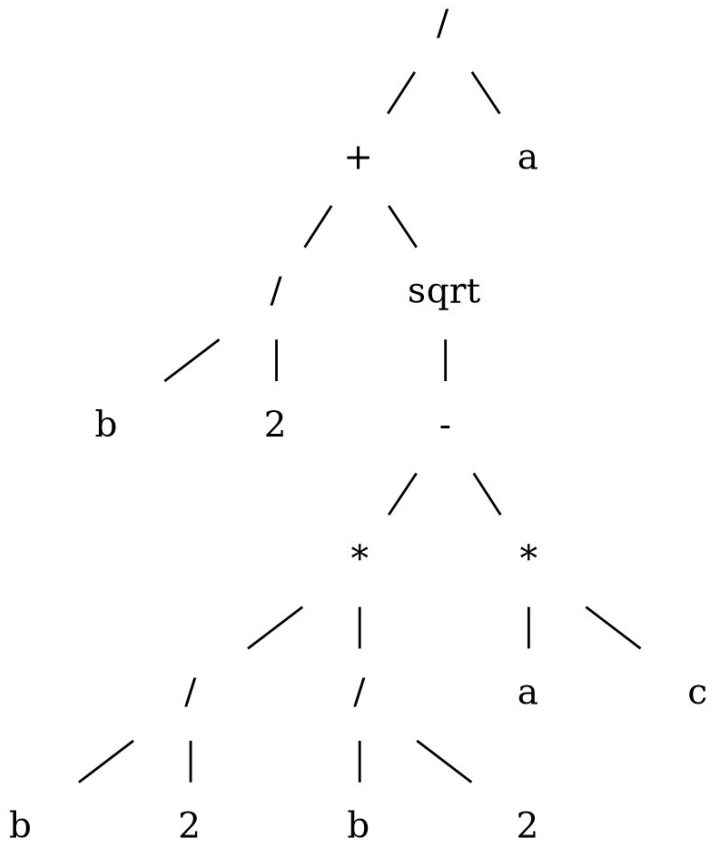


Task 1 Expression 1 AST

Expression 2

(b / 2 + sqrt ((b / 2) * (b / 2) - a * c)) / a

- Prefix notation: / + / b 2 sqrt - * / b 2 / b 2 * a c a
- Postfix notation: b 2 / b 2 / b 2 / * a c * - sqrt + a /
- Abstract syntax tree:



Task 1 Expression 2 AST

Task 2

Write a context-free grammar for strings of letters and/or digits that start with a digit.

Note: the table says “0 length or more”, but an empty string cannot start with a digit, so the grammar below generates strings of length 1 or more.

<S> ::= <digit> <tail>
<tail> ::= <symbol> <tail> | epsilon
<symbol> ::= <letter> | <digit>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<letter> ::= A | B | C | ... | Z | a | b | c | ... | z

Task 3

Using the grammar, show a parse tree and a leftmost derivation for string 1.23456.

<real_number> ::= <integer_part> . <fractional>
<integer_part> ::= <digit> | <integer_part> <digit>
<fractional> ::= <digit> | <digit> <fractional>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Leftmost derivation

<real_number>
=> <integer_part> . <fractional>
=> <digit> . <fractional>
=> 1 . <fractional>
=> 1 . <digit> <fractional>
=> 1.2 <fractional>
=> 1.2 <digit> <fractional>
=> 1.23 <fractional>
=> 1.23 <digit> <fractional>
=> 1.234 <fractional>
=> 1.234 <digit> <fractional>
=> 1.2345 <fractional>
=> 1.2345 <digit>
=> 1.23456

Parse tree

```
                <real_number>
                /     |     \
   <integer_part>     .     <fractional>
         |                   /     \
      <digit>           <digit>  <fractional>
         |                 |        /     \
         1                 2     <digit>  <fractional>
                                    |        /     \
                                    3     <digit>  <fractional>
                                             |        /     \
                                             4     <digit>  <fractional>
                                                      |          |
                                                      5        <digit>
                                                                  |
                                                                  6
```

Task 3 Parse Tree

Task 4

Using the grammar, show a parse tree for the given expressions.

<E> ::= <E> + <T> | <E> - <T> | <T>
<T> ::= <T> * <F> | <T> / <F> | <F>
<F> ::= number | name | (<E>)

Expression 1

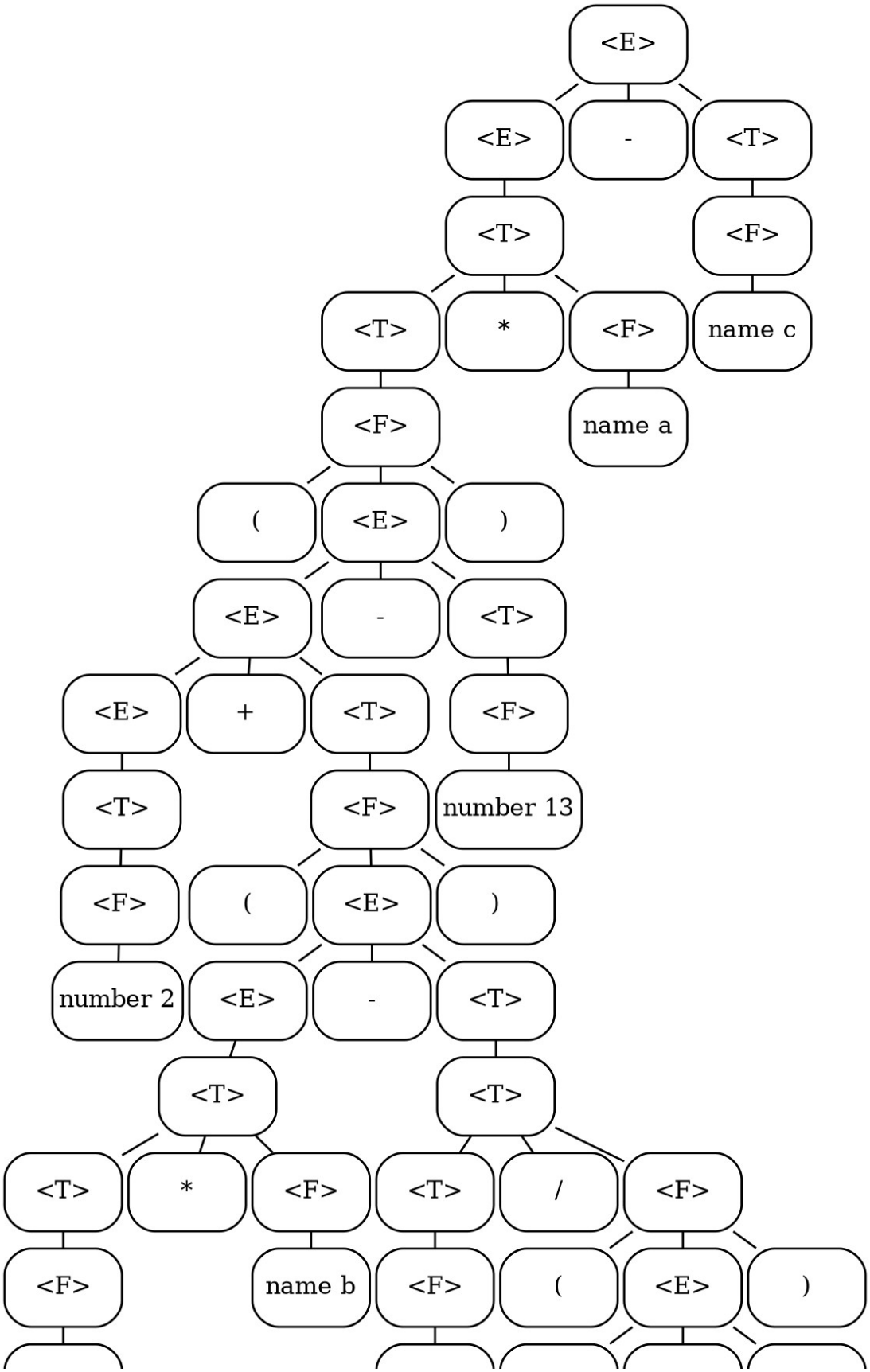
(2 + a) * 5

```
                    <E>
                     |
                    <T>
                 /   |   \
            <T>      *      <F>
             |               |
            <F>           number 5
         /   |   \
     (      <E>      )
         /   |   \
    <E>     +     <T>
     |            |
    <T>          <F>
     |            |
    <F>         name a
     |
  number 2
```

Task 4 Expression 1 Parse Tree

Expression 2

(2 + (a * b - a / (b + 1)) - 13) * a - c


<E>
<E>
-
<T>
<T>
<F>
<T>
*
<F>
name c
<F>
name a
(
<E>
)
<E>
-
<T>
<E>
+
<T>
<F>
<T>
<F>
number 13
<F>
(
<E>
)
number 2
<E>
-
<T>
<T>
<T>
<T>
*
<F>
<T>
/
<F>
<F>
name b
<F>
(
<E>
)

Task 4 Expression 2 Parse Tree

Task 5

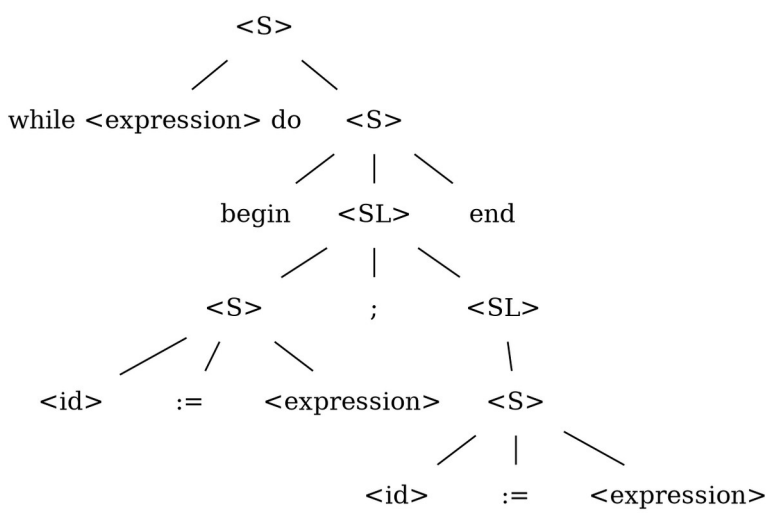
Using the grammar for Pascal language, show a parse tree for the given construction.

<S> ::= <id> := <expression>
| if <expression> then <S>
| if <expression> then <S> else <S>
| while <expression> do <S>
| begin <SL> end
<SL> ::= <S> | <S> ; <SL>

Construction:

while <expression> do begin <id> := <expression>; <id> := <expression> end

Parse tree:



Task 5 Parse Tree